

1KOSMOS

Implemented Logic

React and Django ARR Dashboard POC — Excel import, database persistence, API calculations, filters, and frontend visualization.

Document: 1Kosmos Implemented Logic

Implementation: Booking Database to ARR Dashboard

Frontend: React

Backend: Django

Database: PostgreSQL

Date: 22 June 2026

1. Purpose and Scope

The implemented POC converts the supplied `Booking Database.xlsx` into structured database records and displays calculated ARR information in a React dashboard.

The implementation currently covers:

- Uploading the Booking Database Excel workbook from React.
- Validating the expected Excel template.
- Reading core booking fields and monthly ARR values.
- Saving versioned imports and booking rows in the database.
- Calculating dashboard KPIs, trends, breakdowns, and filters in Django.
- Displaying the API response in the React dashboard.

2. End-to-End Data Flow



3. Main Project Files

File	Responsibility
<code>kosmos_app/models.py</code>	Defines import and booking database tables.
<code>kosmos_app/excel.py</code>	Validates and reads the Booking Database XLSX structure.

File	Responsibility
<code>kosmos_app/views.py</code>	Provides upload and dashboard JSON APIs.
<code>kosmos_app/urls.py</code>	Maps API URLs to Django views.
<code>kosmos/settings.py</code>	Configures Django, PostgreSQL, timezone, hosts, and middleware.
<code>src/pages/Dashboard.js</code>	Uploads Excel, requests API data, manages filters, and renders the dashboard.
<code>src/pages/Dashboard.css</code>	Provides the dashboard layout and visual design.

4. Database Design

4.1 DataImport

This table stores one entry for every successful Excel upload.

Field	Purpose
<code>file_name</code>	Name of the uploaded workbook.
<code>imported_at</code>	Automatic upload date and time.
<code>row_count</code>	Number of imported booking records.

Database table: `kosmos_app_dataimport`

4.2 BookingRecord

This table stores one row for every valid booking line found after the Excel header row. Each row is linked to a `DataImport` through a foreign key.

Important fields include contract details, customer, salesperson, industry, business unit, product, TCV, ARR, booking amount, statuses, term dates, current ARR, and monthly ARR history.

Database table: `kosmos_app_bookingrecord`

Versioning logic: every upload creates a new `DataImport` and new `BookingRecord` rows. The dashboard reads only the newest import, while older imports remain available in the database.

5. Excel Template and Column Mapping

The current parser supports the supplied Booking Database format with these rules:

- The first worksheet is used.
- The header must be on row 22.
- Booking records begin on row 23.
- Cell A22 must contain `Key id`.
- Cell D22 must contain `Contract_ID`.

Excel column	Stored field
A	Key ID
B	Entity
C	Currency
D	Contract ID
E	Contract name
F	Salesperson
G	Mode
H	Company size
I	Industry
J	Business unit
K	Bill-to customer
L	End user
M	Product type
N	Sub-product type
O	Revenue method
Q	TCV USD
S	ARR model value
V	Booking value
W	Booking status
X	Order status
Y	Recurring/non-recurring
Z	Term start
AA	Term end
AD	Line of business

6. Monthly ARR Logic

Columns **AF** through **DP** contain monthly ARR snapshots. Their row-22 values are Excel dates. The parser converts each date to a **YYYY-MM** key and stores each row's monthly values in the **monthly_arr** JSON field.

```
{
  "2026-03": 100000,
  "2026-04": 110000,
  "2026-05": 125000
}
```

The record's `current_arr` is the amount for the latest available month. If no monthly series exists, the parser falls back to the ARR value from column S.

7. How the XLSX Parser Works

An XLSX workbook is a ZIP package containing XML documents. The implementation reads this structure directly, avoiding an additional Excel library.

1. Read the uploaded file into memory.
2. Open it as a ZIP archive.
3. Read `sharedStrings.xml` to resolve stored text values.
4. Resolve and open the first worksheet XML file.
5. Locate row 22 and validate the expected headers.
6. Build the monthly column-to-date mapping.
7. Read all valid rows after row 22.
8. Convert numeric values to floats and Excel dates to Python dates.
9. Build one Python dictionary per booking record.

Excel serial dates are converted from Excel's date base:

```
datetime(1899, 12, 30) + timedelta(days=excel_value)
```

Rows that contain neither a Contract ID nor a Contract Name are skipped.

Formula behavior: this parser does not calculate Excel formulas. It reads values saved inside the workbook. The workbook should be opened, recalculated, and saved in Excel before upload when formulas have changed.

8. Upload API

Endpoint: `POST /api/import/`

React sends the workbook using multipart form data:

```
const formData = new FormData();
formData.append("file", file);
fetch("/api/import/", { method: "POST", body: formData });
```

Django verifies that a file exists, its extension is `.xlsx`, the ZIP/XML structure is valid, the template headers match, and at least one booking record exists.

Database writing occurs inside `transaction.atomic()`. Therefore, a failed operation rolls back the complete upload instead of saving partial data. Records are inserted with `bulk_create` in batches of 250 for better performance.

9. Dashboard API and Calculations

Endpoint: `GET /api/dashboard/`

The latest DataImport is selected because the model is ordered by newest `imported_at`. Only records connected to that import are included.

Metric	Implemented calculation
Closing ARR	Sum of <code>current_arr</code> for selected records.
Total bookings	Sum of the imported booking values.
Customers	Unique count using End User, with Bill To as fallback.
Recurring contracts	Count of rows where revenue type equals Recurring.
Average ARR	Total current ARR divided by recurring contract count.
Monthly trend	Sum every record's monthly ARR amount for each month.
Breakdowns	Group and sum current ARR by business unit, industry, customer, or sub-product.
Contract table	Sort by current ARR descending and return the first 50 records.

10. Filter Logic

The API supports business unit, industry, salesperson, and product type filters. React sends them as URL query parameters:

```
GET /api/dashboard/?business_unit=NAM&industry=TMT
```

Django filters the latest import's queryset before calculating any values. Therefore, KPI cards, trends, breakdowns, and table rows all represent the same selected data.

11. React Dashboard Logic

The React dashboard manages:

- `data` — current dashboard API response.
- `filters` — selected filter values.
- `loading` — dashboard request state.
- `uploading` — workbook import state.

- `message` — success or error feedback.

When the page loads, React requests the dashboard API. When a filter changes, React rebuilds the query string and requests recalculated data. After a successful upload, filters are cleared and the dashboard is refreshed.

The UI includes KPI cards, an SVG monthly trend, proportional bar lists, and a largest-contract table. Currency values use the browser's USD formatter, including compact million notation.

12. API Summary

Method	URL	Purpose
POST	<code>/api/import/</code>	Validate, parse, and save the Booking Database workbook.
GET	<code>/api/dashboard/</code>	Return the latest dashboard data.
GET	<code>/api/dashboard/?business_unit=NAM</code>	Return recalculated filtered data.

13. Local Execution

Backend

```
cd kosmos
.\env\Scripts\python.exe manage.py migrate
.\env\Scripts\python.exe manage.py runserver
```

Frontend

```
cd kosmos_frontend
npm.cmd start
```

Open `http://localhost:3000` and upload `requirements\Booking Database.xlsx`.

14. Current POC Limitations

- Only the Booking Database workbook is imported.
- Pipeline, AR Aging, Payment History, and Renewal Pending are not integrated.
- Azure AD SSO and production role-based access are not implemented.
- No Excel/CSV export or import-history user interface exists.
- Older versions remain in the database but cannot yet be selected from React.
- Duplicate uploads are not currently blocked.
- Large uploads run synchronously rather than through background jobs.
- Excel formulas are not recalculated by the backend.

15. Recommended Next Phase

1. Add an import history screen and version activation/rollback.
2. Add detailed row-level validation and downloadable error reports.
3. Integrate Pipeline and AR Dashboard workbooks through separate models.
4. Add authentication, admin/consumer roles, and scoped data access.
5. Add filtered Excel export.
6. Move credentials to environment variables and configure production deployment.

Generated from the implemented 1Kosmos React and Django POC on 22 June 2026.